



Resource



07. Resource

Resource Manager

게임 프로그래머 | 이창진

| 목차

Chapter 1.

Resource Manager의 필요성

Chapter 2.

경로기반, 데이터 기반 해싱 차이

Chapter 3.

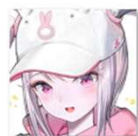
애셋매니저

Resource Manager

정의

엔진에서 데이터를 로드할때 이를 관리하는 매니저

게임에서 쓰이는 데이터들에 대한 정보를 미리 저장하고
이를 매니저가 관리하는 방식



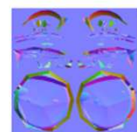
Alice.ico



Alice_fbx



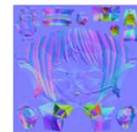
head_eye_M_BC.jpg



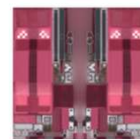
head_eye_M_BC_normal.png



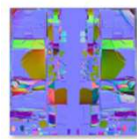
head_face_M_BC.jpg



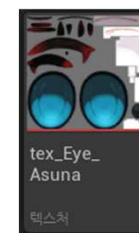
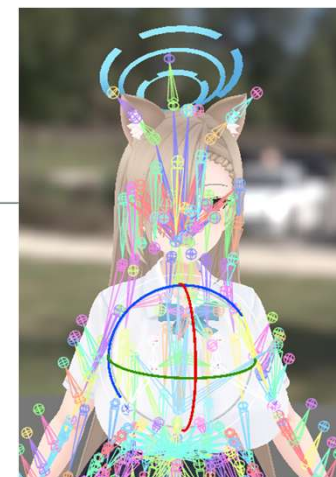
head_face_M_BC_normal.png



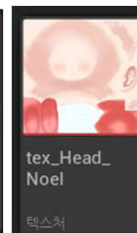
bag_M_BC.jpg



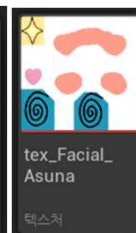
bag_M_BC_normal.png



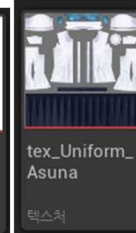
텍스처



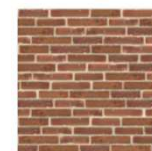
텍스처



텍스처



텍스처



Bricks059_1K-JPG_Color.jpg



Bricks059_1K-JPG_NormalDX.jpg



Bricks059_Specular.png

Chapter 1.

Resource Manager의 필요성

필요성

씬에 있는 인스턴스가 중복된 애셋을 사용할 때 같은 애셋을 사용하지만 새로운 인스턴스를 생성할 경우 매번 데이터를 로드해서 사용하면 비용이 높아집니다. (걸리는 시간, 메모리 둘다.)

이를 보완하기 위해 **최초 로드한 애셋에 대한 정보를 캐싱**해둡니다. 애셋을 로드하려고 하면 캐싱된 정보를 먼저 탐색하고 캐시 데이터가 있다면 반환하면 됩니다.



최초 로드된 아스나



최초 로드 이후 아스나를 요청하면

	xb5	x04	...

메모리 공간에 캐싱되어 있는 아스나를 가져옴

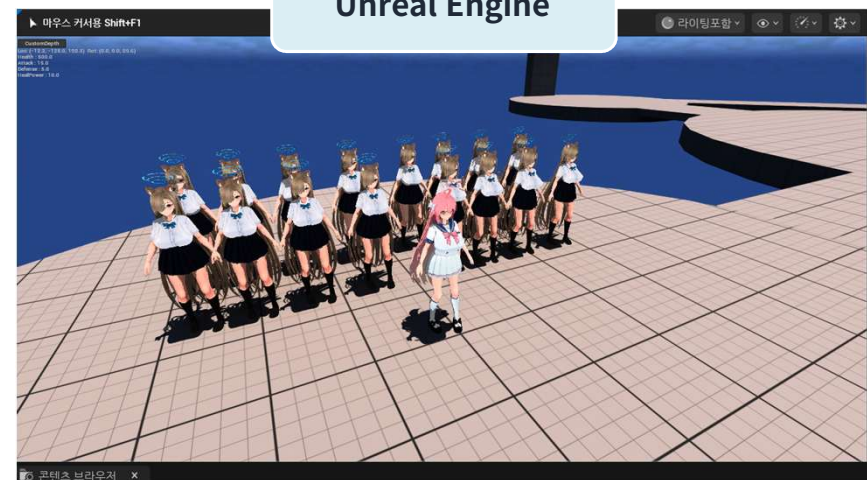


shared_ptr, weak_ptr로 복제된 아스나

Dx11



Unreal Engine



경로기반, 데이터 기반 해싱 차이

정의

2 개로 나뉩니다. 경로기반, 데이터 기반

1. 키로 파일의 경로를 넣는 해싱
2. 키로 데이터의 바이너리를 넣는 해싱

애셋을 로드하려고 할때 그 애셋의 경로를 기반으로 해시를 만들어 애셋 매니저의 key값으로 넣는 방법은 쉽지만 경로가 바뀌면 키도 바뀌고 문자열 비교라 느립니다.

반면 데이터의 바이너리 값 자체를 넣으면 실제 데이터를 쿠키нг하기 때문에 경로가 바뀌어도 동일 리소스로 인식합니다. 또한 정수 비교이므로 매우 빠릅니다.

Fbx 파일에 대한 캐싱을 지원합니다.

```
class AssetManager : public Singleton<AssetManager>
{
public:
    AssetManager() = default;

    std::shared_ptr<FbxModel> GetFbxModel(ID3D11Device* device, const std::wstring& pathW);

private:
    std::unordered_map<AssetKey, std::weak_ptr<FbxModel>, AssetKey::Hash> m_FbxCache;;
```

경로기반

```
class AssetManager : public Singleton<AssetManager>
{
public:
    AssetManager() = default;
    std::shared_ptr<FbxModel> GetFbxModel(ID3D11Device* device, const std::wstring& pathW);

private:
    static uint64_t ComputeFileHash(const std::wstring& pathW);
    std::unordered_map<AssetKey, std::weak_ptr<FbxModel>, AssetKey::Hash> m_FbxCache;
    std::unordered_map<std::wstring, uint64_t> m_PathToHash;
    std::mutex m_Mutex;
};
```

데이터기반

1. 경로기반 해싱

정의

경로기반 해싱에서는 단순히 파일의 경로를 사용합니다.

경로를 key 값으로 넣기때문에 스트링 비교 연산을 매번 수행합니다

Fbx 파일에 대한 캐싱을 지원합니다.

```
enum class EAssetKind
{
    FbxModel,
    ObjModel,
    PmxModel,
};

struct AssetKey
{
    std::wstring PathW;
    EAssetKind Kind;

    bool operator==(const AssetKey& other) const
    {
        return Kind == other.Kind && PathW == other.PathW;
    }
};

class AssetManager : public Singleton<AssetManager>
{
public:
    AssetManager() = default;
    std::shared_ptr<FbxModel> GetFbxModel(ID3D11Device* device, const std::wstring& pathW);

private:
    std::unordered_map<AssetKey, std::weak_ptr<FbxModel>, AssetKey::Hash> m_FbxCache;
};
```

1. 경로기반 해싱

정의

Assetkey로 캐시를 조회합니다

캐시가 이미 있다면 shared_ptr를 리턴합니다.
없다면 새로 로드를 하고 리턴합니다.



shared_ptr



최초 로드 이후 아스나를 요청하면

	xb5	x04	...

메모리 공간에 캐싱되어 있는
아스나를 가져옴



weak_ptr

```
std::shared_ptr<FbxModel> AssetManager::GetFbxModel(ID3D11Device* device, const
std::wstring& pathW)
{
    if (!device || pathW.empty()) return nullptr;

    // AssetKey로 캐시 조회
    AssetKey key{ pathW, EAssetKind::FbxModel };
    if (auto it = m_FbxCache.find(key); it != m_FbxCache.end())
    {
        if (auto sp = it->second.lock())
        {
            return sp; // 재사용
        }
        else
        {
            m_FbxCache.erase(it); // 만료된 weak_ptr 정리
        }
    }

    // 새로 로드
    auto model = std::make_shared<FbxModel>();
    if (!model->Load(device, pathW))
    {
        return nullptr;
    }
    m_FbxCache[key] = model;
    return model;
}
```

2. 데이터기반 해싱 – FNV-1a

정의

비암호화 해시 함수

암호화가 아닌 빠른 해시 테이블과 체크섬을 위해 설계된 알고리즘입니다.

XOR을 먼저하고 곱셈을 합니다. 이때 소수 곱셈을 사용합니다. 분산을 좋게 만드는 겁니다. 64비트의 오버플로우를 사용해서 적당하게 비트가 섞이도록 합니다.

곱셈 전에 XOR을 사용하는데 이로 인해 관련 입력의 해시 입력 값 사이에 예측 가능한 관계나 패턴이 생길수 있는 문제가 있습니다.

서로 다른 값 (a : b), (1 : 2)가 동일한 비트 패턴을 공유하기 때문에 입력의 특정 비트 대칭이 의도치 않은 해시 상관관계를 만들어 낼 수 있습니다. 즉 똑같은 패턴이 나온다는 이야기입니다.

밑의 사이트에 들어가서 다양한 해시 함수를 만나보세요

• “a” 문자 변환 과정

hash = FNV_OFFSET_BASIS = 14695981039346656037 = 0xCBF29CE484222325

1단계. XOR 연산 (hash ^= byte)

hash = hash ^ data[0] = 0xCBF29CE484222325 ^ 0x61 = 0xCBF29CE484222344
 0xCBF29CE484222325 (초기 해시값)
 ^ 0x0000000000000061 (문자 'a' = 97)

0xCBF29CE484222344 (XOR 결과)

2단계. 곱셈 연산 (hash = FNV_PRIME)

hash = hash * FNV_PRIME = 0xCBF29CE484222344 * 0x100000001B3
 0xCBF29CE484222344 (XOR 결과)
 * 0x00000100000001B3 (FNV_PRIME)

(64비트 오버플로우 발생, 하위 64비트만 유지)

hash ≈ 0xAF63DC4C8601EC8C

3단계. 10진수로 변환하기

hash = 0xAF63DC4C8601EC8C = 12638153115695167496 (10진수)

2. 데이터기반 해싱

정의

데이터기반 해싱에서는 여러 데이터를 해싱 계산에 포함합니다.

FNV-1a는 구조적으로 해시의 결과가 비슷하게 나올 수 있으므로 파일크기, 수정시간을 같이 추가해서 넣었습니다.

경로기반에서 이 해시를 키로 넣으면 됩니다.

1. 파일크기
2. 수정시간
3. 데이터 바이너리의 첫 4KB, 끝 4KB
4. FNV-1a 해시
5. 1~4번을 xor해서 최종 해시

```
uint64_t AssetManager::ComputeFileHash(const std::wstring& pathW)
{
    if (!std::filesystem::exists(pathW)) return 0;
    auto fileSize = std::filesystem::file_size(pathW);
    auto writeTime = std::filesystem::last_write_time(pathW);
    std::uint64_t timeHash = static_cast<std::uint64_t>(writeTime.time_since_epoch().count());
    uint64_t sizeHash = fileSize;

    const size_t SAMPLE_SIZE = 4096;
    std::ifstream file(pathW, std::ios::binary);
    if (!file.is_open()) return 0;
    uint8_t buffer[SAMPLE_SIZE * 2] = { 0 };
    size_t readSize = 0;
    if (fileSize > 0)
    {
        size_t toRead = (std::min)(SAMPLE_SIZE, static_cast<size_t>(fileSize));
        file.read(reinterpret_cast<char*>(buffer), toRead);
        readSize += static_cast<size_t>(file.gcount());
    }
    if (fileSize > SAMPLE_SIZE)
    {
        file.seekg(-static_cast<std::streamoff>((std::min)(SAMPLE_SIZE, static_cast<size_t>(fileSize))), std::ios::end);
        file.read(reinterpret_cast<char*>(buffer + readSize), SAMPLE_SIZE);
        readSize += static_cast<size_t>(file.gcount());
    }
    file.close();
    uint64_t sampleHash = FNV1a64(buffer, readSize);
    uint64_t finalHash = sizeHash;
    finalHash ^= (timeHash << 1);
    finalHash ^= (sampleHash << 2);

    return finalHash;
}
```

감사합니다.